2주차_4장

4장. 딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

- 퍼셉트론
  - 딥러닝의 기원이 되는 알고리즘
  - 다수의 신호를 받아 하나의 신호를 출력 (전달:1 / 무시:0)

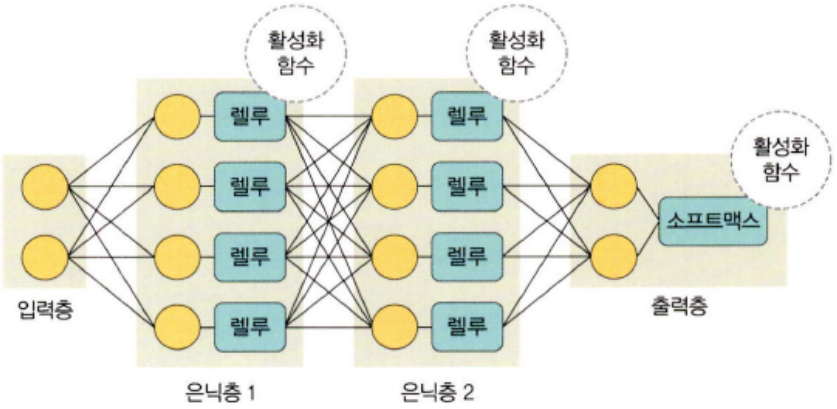
1. AND 게이트: 모든 입력이 1일 때 작동
2. OR 게이트: 두 입력 중 하나만 1이거나 둘 다 1일 때 작동
3. XOR 게이트(=배타적 논리합): 둘 중 하나만 1일 때 작동
   - 데이터가 비선형적으로 분리되므로 단층 퍼셉트론에서 학습이 불가능

⇒ 은닉층을 가진 다층 퍼셉트론이 고안됨.
⇒ 은닉층이 여러 개 있는 신경망을 심층 신경망(=딥러닝)이라고 함.

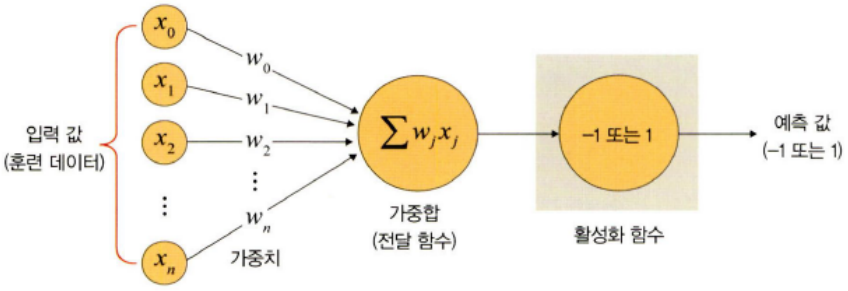
4.2 딥러닝 구조

[용어]

1. 층
   a. 입력층: 데이터를 받아들이는 층
   b. 은닉층: 모든 입력 노드로부터 입력 값을 받아 가중합을 계산하고, 이를 활성화 함수에 적용하여 출력층에 전달하는 층
   c. 출력층: 신경망의 최종 결괏값이 포함된 층



2. 가중치: 노드 간의 연결 강도, 각 입력 값이 연산에 미치는 영향력을 조절



3. 가중합 / 전달 함수: 가중치와 신호의 곱을 합한 것

$$\sum_i w_i x_i + b$$

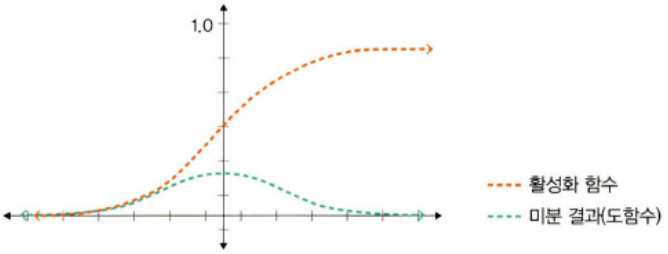
4. 바이어스: 가중합에 더해 주는 상수, 하나의 뉴런에서 출력 값을 조절함

(w: 가중치, b: 바이어스)

5. 활성화 함수: 전달 함수에서 값을 입력받아 처리하여 출력해주는 비선형 함수
   a. 시그모이드 함수

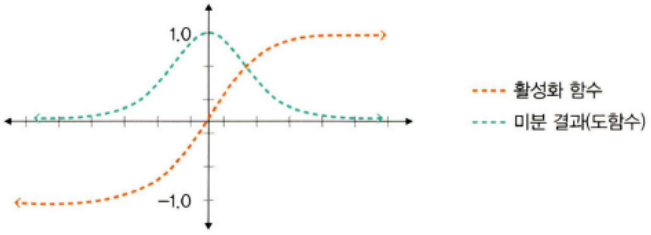
$$f(x) = \frac{1}{1+e^{-x}}$$

- 0 ~ 1 사이 값으로 변환
- 분류 문제를 확률적으로 표현할 때 사용
- 층이 깊어지면 기울기 소멸 문제 → 딥러닝에서 잘 사용 X



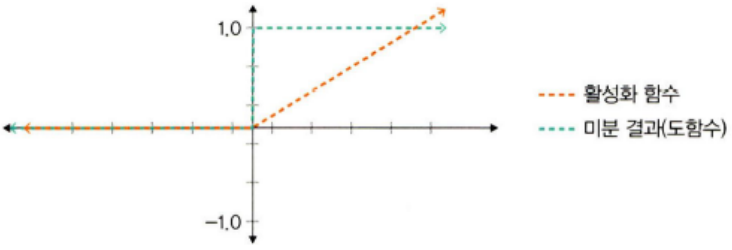
b. 하이퍼볼릭 탄젠트 함수

- -1 ~ 1 사이 값으로 변환
- 시그모이드의 평균이 편향된 문제를 해결
- 기울기 소멸 문제 여전히 발생



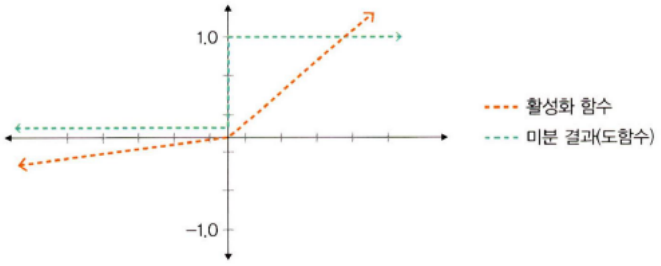
c. 렐루 함수

- x가 음수일 때는 0을, 양수일 때는 x를 출력
- 경사 하강법에 영향 X → 학습 속도 빠름
- 기울기 소멸 문제 X
- 일반적으로 은닉층에서 사용



d. 리키 렐루 함수

- x가 음수일 때 0을 반환하는 것이 아닌 매우 작은 수를 곱해 반환
- 출력 값이 수렴하여 학습 능력이 감소하는 문제 해결



e. 소프트맥스 함수

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_i)}$$

- 0 ~ 1 사이로 출력, 출력 값들의 총합이 항상 1이 되도록 함.
- 출력 노드의 활성화 함수로 많이 사용

```
# 렐루 함수와 소프트맥스 함수 구현
class Net(torch.nn.Module):
    def __init__(self,n_feature,n_hidden,n_output):
        super(Net,self).__init__()
        self.hidden=torch.nn.Linear(n_feature,n_hidden)
        self.relu=torch.nn.ReLU(inplace=True)
        self.out=torch.nn.Linear(n_hidden,n_output)
        self.softmax=torch.nn.Softmax(dim=n_output)
    def forward(self,x):
        x=self.hidden(x)
        x=self.relu(x)
        x=self.out(x)
        x=self.softmax(x)
        return x
```

6. 손실 함수: 가중치 학습을 위해 오차를 측정하는 함수
   → 경사하강법: 학습률과 손실 함수의 미분값을 이용해 가중치를 업데이트

   a. 평균 제곱 오차: 실제 값과 예측 값의 차이를 제곱해 평균을 낸 것

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i\text{: 신경망의 출력(신경망이 추정한 값)} \\ y_i\text{: 정답 레이블} \\ i\text{: 데이터의 차원 개수} \end{array}\right)$$

- 회귀에서 주로 사용

```
import torch
loss_fn=torch.nn.MSELoss(reduction='sum')
y_pred=model(x)
loss=loss_fn(y_pred,y)
```

b. 크로스 엔트로피 오차

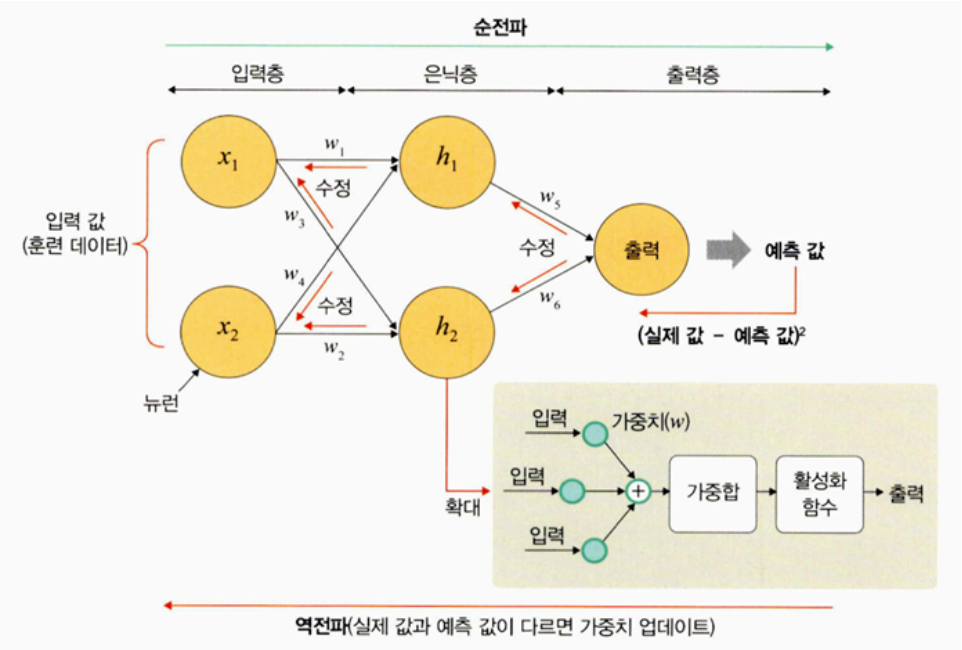
- classification에서 원-핫 인코딩 했을 때만 사용
  → 시그모이드 함수의 자연 상수 e 때문에 평균 제곱 오차 사용 시 울퉁불퉁한 그래프가 되기 때문

$$CrossEntropy = -\sum_{i=1}^{n} y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i\text{: 신경망의 출력(신경망이 추정한 값)} \\ y_i\text{: 정답 레이블} \\ i\text{: 데이터의 차원 개수} \end{array}\right)$$

```
loss=nn.CrossEntropyLoss()
input=torch.randn(5,6,requires_grad=True)
target=torch.empty(3,dtype=torch.long).random_(5)
output.backward()
```

[딥러닝 학습]



1. <u>순전파</u>:
   - 네트워크에 훈련 데이터가 들어올 때 발생
   - 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환을 적용하여 다음 층의 뉴런으로 전송

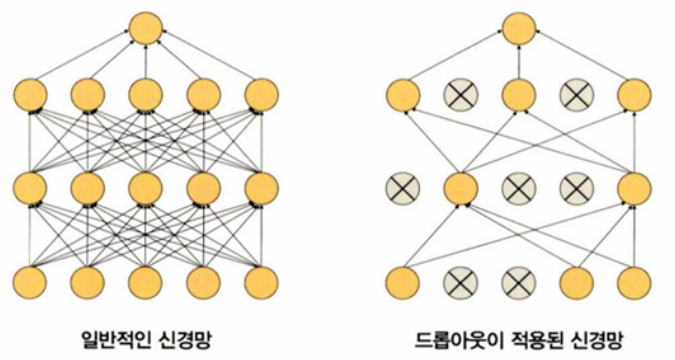
   ⇒ 예측 값이 출력층에 도달
2. 손실 함수로 오차를 추정
3. <u>역전파:</u>
   - 오차 정보가 역으로 전파됨 (출력층 → 은닉층 → 입력층)
   - 오차를 각 뉴런의 가중치로 미분
4. 기존 가중치 값에서 미분값을 뺀 결과를 다시 순전파의 가중치 값으로 사용

[딥러닝의 문제점]

은닉층 개수가 많을수록 분류가 잘 되지만 세 가지 문제점이 생김.

1. <u>과적합 문제</u>
   - 훈련 데이터를 과하게 학습해서 발생
   - 검증 데이터에 대해서 오차가 증가

   ⇒ 해결 방법: **드롭 아웃**
   - 임의로 일부 노드들을 학습에서 제외



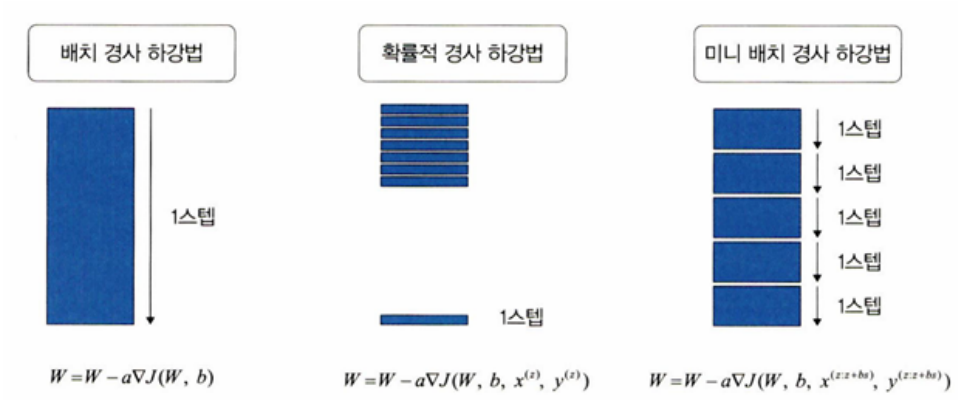

```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0.5)  # 50%의
노드를 무작위로 선택해 사용하지 않음
        self.layer2 = torch.nn.Linear(1200, 1200)
        self.dropout2 = torch.nn.Dropout(0.5)
        self.layer3 = torch.nn.Linear(1200, 10)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self.layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

2. <u>기울기 소멸 문제</u>
   - 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
   - 즉 학습되는 양이 0에 가까워져 오차를 더 줄이지 못하고 그 상태로 수렴

   ⇒ 해결 방법: 시그모이드, 하이퍼볼릭 탄젠트 대신 **렐루 활성화 함수를 사용**
3. <u>성능 저하 문제</u>
   - 경사하강법에서 성능이 나빠지는 문제 발생

   ⇒ 해결 방법: 확률적 경사 하강법, 미니 배치 경사 하강법



a. 배치 경사 하강법 (BGD)

$$W = W - a\nabla J(W, b)$$
$$(a\text{: 학습률}, J\text{: 손실 함수})$$

- 전체 훈련 데이터셋에 대해 가중치를 편미분 → 학습이 오래 걸림

b. 확률적 경사 하강법 (SGD)

- 임의로 선택한 데이터에 대해 기울기를 계산 → 파라미터 변경 폭이 불안정하고 정확도가 낮을 수 있지만 속도가 빠르다.

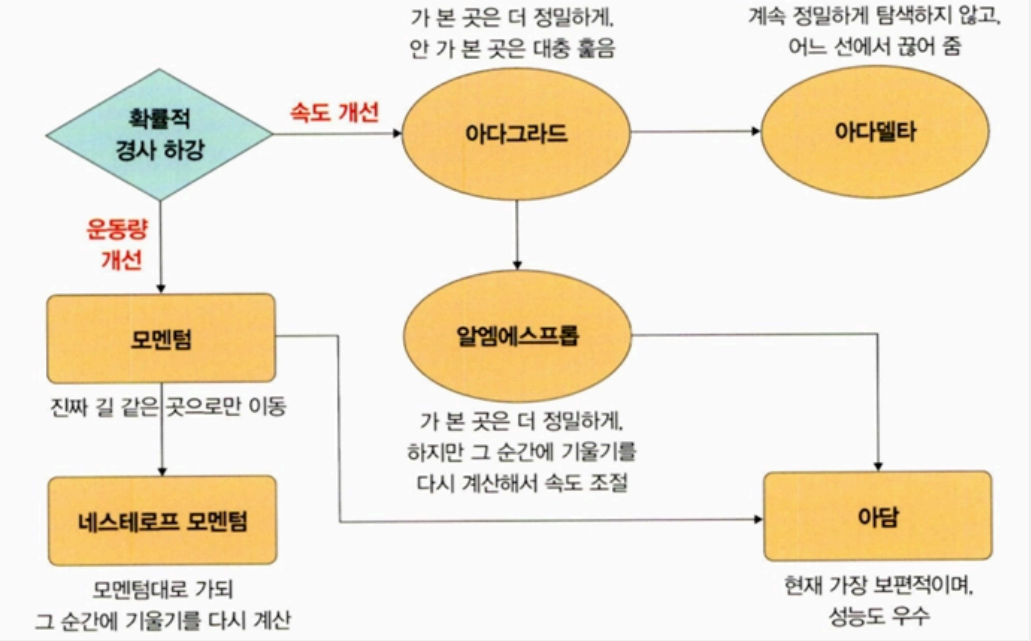
c. 미니 배치 경사 하강법

- 데이터셋을 미니 배치 여러 개로 나누고 한 개마다 기울기를 구해 평균 기울기로 모델을 업데이트 → 배치 경사 하강법보다 빠르고 확률적 경사 하강법보다 안정적이라 가장 많이 사용.

```
class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
        self.y_data = [[12], [18], [11]]
    def __len__(self):
        return len(self.x_data)
    def __getitem__(self, idx):
        x = torch.FloatTensor(self.x_data[idx])
        y = torch.FloatTensor(self.y_data[idx])
        return x, y
dataset = CustomDataset()
dataloader = DataLoader(
    dataset,
```

```
    batch_size=2,   # 미니 배치 크기
    shuffle=True)    # 데이터를 랜덤으로 섞음
```

💡 **옵티마이저**: 학습 속도와 운동량 조정
→ 확률적 경사 하강법에서 파라미터 변경 폭이 불안정한 문제 해결



[딥러닝을 사용할 때 이점]

1. 딥러닝에서는 특성 추출 과정을 알고리즘에 통합시켜 데이터에 대한 전문 지식이 필요 없음.
2. 딥러닝 학습을 이용한 특성 추출은 데이터 사례가 많을수록 성능이 향상됨

4.3 딥러닝 알고리즘

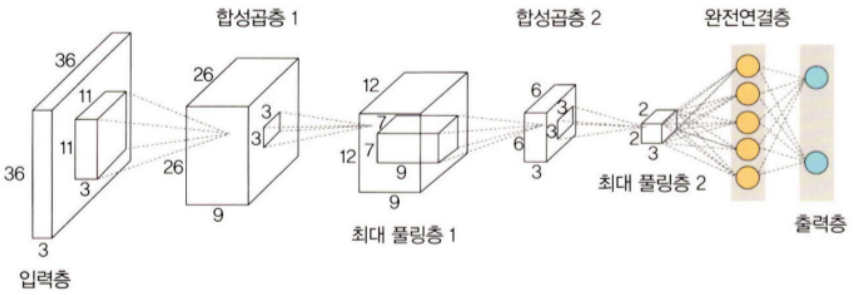
[심층 신경망(DNN)]

: 다수의 은닉층을 포함 → 다양한 비선형적 관계를 학습할 수 있음.

- 연산량이 많고 기울기 소멸 문제 등이 발생

⇒ 드롭아웃, 렐루 함수, 배치 정규화 등을 적용

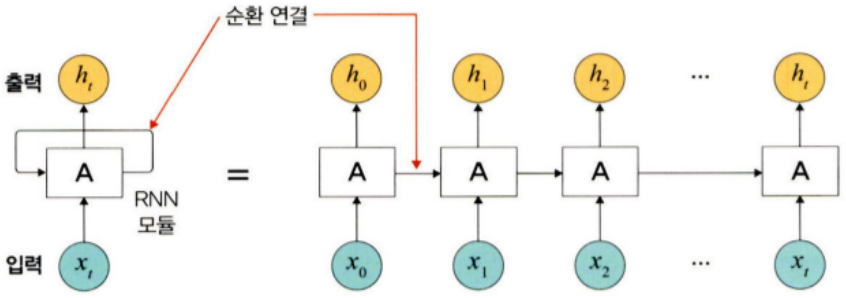
[합성곱 신경망(CNN)]



: 합성곱과 풀링층을 포함, 이미지 데이터에서 객체를 탐색하는 데 유용

- 기존 신경망과의 차별성:
  - 각 층의 입출력 형상을 유지함.
  - 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
  - 복수 필터로 이미지의 특징을 추출하고 학습
  - 추출한 이미지의 특징을 모으고 강화하는 **풀링층**이 있음
  - 필터를 공유 파라미터로 사용하기 때문에 학습 파라미터가 매우 적음

[순환 신경망(RNN)]



: 시계열 데이터를 학습하기 위한 인공 신경망

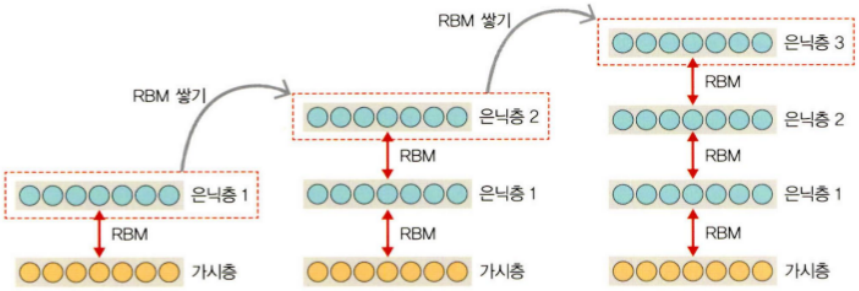
- 기울기 소멸 문제 → LSTM 많이 사용 (메모리 개념 도입)
- 자연어 처리 분야와 잘 맞음

[제한된 볼츠만 머신]

: 가시층과 은닉층으로 구성된 모델, 같은 층 간에 연결이 없음.

- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용
- 심층 신뢰 신경망의 요소로 활용

[심층 신뢰 신경망(DBN)]



: 제한된 볼츠만 머신을 여러 층으로 쌓은 형태로 연결된 신경망

- 비지도 학습이 가능
- 이미지 일반화와 추상화 과정에서 유용
- 학습 절차:
  1. 가시층과 은닉층1에 제한된 볼츠만 머신을 사전 훈련
  2. 첫 번째 층 입력 데이터와 파라미터를 고정해 두 번째 층 제한된 볼츠만 머신을 사전 훈련
  3. 원하는 층 개수만큼 쌓아 올려 전체 DBN을 완성
- 특징:
  - 위로 올라갈수록 추상적 특성을 추출함.
  - 학습된 가중치를 다층 퍼셉트론의 가중치 초깃값으로 사용